Wiseliy Repository Branching & Version Control Guidelines

Introduction 3
1. Branching Strategy Overview 4
2. Standard Branch Structure 5
2.1 Dev Branch 5
Purpose 5
Guidelines 5
2.2 Release Branch 6
Purpose 6
Guidelines 6
2.3 Master/Main Branch 7
Purpose 7
Guidelines 7
3. Feature Branch Strategy 7
3.1 Feature Branch Creation 7
3.2 Feature Development Process 8
3.3 Parallel Feature Development 8
4. Release Management Process 9
4.1 Release Preparation 9
4.2 QA/Staging Deployment 9
4.3 Release Approval 10
5. Production Deployment Process 10
6. Hotfix Branch Strategy 11
6.1 Hotfix Branch Creation 11
6.2 Hotfix Development Process 12
6.3 Hotfix Synchronization 12
7. Branch Flow Summary 12
7.1 Normal Feature and Release Flow 13
7.2 Hotfix Flow 13
8. Pull Request Policy 14

8.1 General Requirements 14
8.2 Pull Request Description 14
9. Commit Guidelines 15
Recommended practices 15
10. Branch Naming Convention 15
11. Code Review Best Practices 16
12. Merge and Conflict Management 17
13. Environment and Branch Alignment 18
14. Release Version and Traceability 18
15. Database Change Management 19
16. Configuration and Secret Management 19
17. Testing Requirements 20
Feature Development 20
Dev 20
Release 20
Production 21
18. Developer Pre-Merge Checklist 21
19. Release Checklist 21
20. Hotfix Checklist 22
21. General Git Best Practices 22
22. Branch Protection Recommendations 24
Dev 24
Release 24
Master/Main 24
23. Roles and Responsibilities 25
Developers 25
Reviewers 25
QA Team 25
Release/Deployment Team 26

24. Exception Handling 26
25. Branching Lifecycle Summary 26
Standard Development 26
Release 27
Production 27
Production Hotfix 27
Hotfix Synchronization 27
26. Guiding Principle 27

Introduction

This document defines the standard Git branching and release management guidelines to be followed across all software development projects. The purpose is to establish a consistent approach for managing source code changes, feature development, bug fixes, releases, and production deployments.

Following these guidelines will help maintain code stability, improve collaboration between development and QA teams, provide clear traceability of changes, and reduce the risk of untested or incomplete code being deployed to production.

All developers working on the project repository are expected to follow the branching, pull request, code review, merging, and release practices defined in this document.

1. Branching Strategy Overview

Wiseliy follows a structured Git branching strategy designed to separate ongoing development, release preparation, quality assurance, and production-ready code.

The standard branching model consists of the following branches:

- **Dev** – Main development and integration branch.
- **Release** – Release candidate branch used for QA/Staging validation.
- **Master/Main** – Production-ready branch used for production deployments.
- **Feature** – Temporary branches created for development of individual features or enhancements.
- **Hotfix** – Temporary branches created directly from the Master/Main branch to address critical production issues.

This branching approach provides a controlled flow of code:

Feature → Dev → Release → Master/Main → Production

For production defects where the normal development branches contain unreleased changes, the flow is:

Master/Main → Hotfix → Release/QA → Master/Main → Production

The exact branch name may be **master** or **main** depending on the repository configuration. Throughout this document, **Master/Main** refers to the repository's designated production branch.

2. Standard Branch Structure

2.1 Dev Branch

The **Dev** branch is the primary development and integration branch.

It contains the latest integrated development changes that are intended to become part of a future release.

Purpose

- Serve as the central integration branch for ongoing development.
- Receive completed feature branches through Pull Requests.
- Allow multiple teams or developers to work on different features concurrently.
- Provide a common branch for integration testing before changes are promoted to a release.
- Contain development work that may not yet be ready for production.

Guidelines

- Developers should not directly commit changes to the Dev branch unless explicitly permitted by the repository policy.
- Changes should normally be introduced through Feature branches and Pull Requests.
- Code merged into Dev should compile/build successfully and should not knowingly break existing functionality.

- Developers should keep their Feature branches synchronized with Dev where appropriate to minimize merge conflicts.
- Dev should not be considered production-ready.

2.2 Release Branch

The **Release** branch represents a specific release candidate and is used for QA/Staging validation.

Changes are promoted from Dev into the Release branch when the development scope for a release is considered ready for formal QA testing.

Purpose

- Provide a stable branch for QA/Staging deployment.
- Isolate a release candidate from ongoing development work in Dev.
- Allow QA to test one or more completed features together.
- Provide a controlled environment for release-specific bug fixes and validation.
- Ensure that only tested and approved changes are promoted to production.

Guidelines

- The Release branch should be created or updated from Dev according to the project's release process.
- The Release branch should be deployed to the QA/Staging environment.
- QA should validate the functionality, integration, regression scenarios, and relevant non-functional requirements.
- Any defects identified during QA should be fixed and re-tested before the release is approved.
- Release-specific fixes should be carefully managed to ensure they are also reflected in the appropriate development branch.
- Once all required testing and approvals are completed, the Release branch can be merged into Master/Main.
- The Release branch must not be merged into Master/Main while known blocking defects remain unresolved.

2.3 Master/Main Branch

The **Master/Main** branch is the production-ready branch and represents the version of the code that is approved for production deployment.

Purpose

- Maintain the production-ready source code.
- Provide a reliable and traceable representation of what is deployed to production.
- Serve as the source branch for production Hotfixes.
- Maintain the history of approved releases.

Guidelines

- Direct commits to Master/Main should not be permitted.
- All changes should be introduced through an approved Pull Request.
- Only fully tested and approved Release branches or approved Hotfix branches should be merged into Master/Main.
- Production deployments should be performed from the approved Master/Main commit.
- Every production release should have a clear version, tag, release number, or other traceable identifier where applicable.
- The branch should always remain in a deployable state.

3. Feature Branch Strategy

Feature branches are temporary branches used to develop individual features, enhancements, user stories, or other planned changes.

3.1 Feature Branch Creation

Feature branches should be created from the **Dev** branch.

Example:

Dev → Feature Branch

Multiple Feature branches can exist simultaneously, allowing different developers or teams to work independently.

Example:

- `feature/user-notifications`
- `feature/payment-integration`
- `feature/report-export`
- `feature/new-dashboard`

The actual naming convention should follow the repository or project-specific naming standard.

3.2 Feature Development Process

The standard Feature development process is:

1. Create the Feature branch from the latest appropriate Dev branch.
2. Implement the required functionality.
3. Perform local development testing.
4. Commit changes using meaningful commit messages.
5. Keep the Feature branch synchronized with Dev when necessary.
6. Resolve merge conflicts before raising or completing the Pull Request.
7. Raise a Pull Request from the Feature branch to Dev.
8. Complete the required code review and automated checks.
9. Address all review comments and CI/CD failures.
10. Merge the approved Feature branch into Dev.
11. Delete the Feature branch after successful merge where repository policy permits.

3.3 Parallel Feature Development

Multiple Feature branches may be developed in parallel.

For example:

Dev

↓

Feature A → Dev

Feature B → Dev

Feature C → Dev

This allows multiple teams to work independently without directly interfering with each other's development work.

However, developers must ensure that changes are properly integrated and tested after merging into Dev.

4. Release Management Process

Once the required development scope has been completed and is ready for QA, the changes are promoted from Dev to the Release branch.

The standard release flow is:

Feature → Dev → Release → QA/Staging → Master/Main → Production

4.1 Release Preparation

Before promoting changes to the Release branch, the development team should verify:

- Required features for the release are completed.
- Relevant Feature branches have been merged into Dev.
- Code builds successfully.
- Automated tests are passing.
- Known development issues have been reviewed.
- Required configuration changes have been identified.
- Required database scripts or migration changes have been prepared.
- Dependencies and package changes have been reviewed.
- Release notes or deployment documentation have been prepared where required.

4.2 QA/Staging Deployment

The Release branch is deployed to the QA/Staging environment.

QA validates the release against the agreed requirements and test scenarios.

Testing may include:

- Functional testing.
- Integration testing.
- Regression testing.
- API testing.
- UI testing.
- Database validation.
- Permission/access validation.
- Notification and background process validation.
- Configuration validation.
- Relevant performance or security validation.

Any defects identified during QA should be documented, assigned, fixed, and re-tested.

4.3 Release Approval

A Release should only be promoted to Master/Main after:

- Required QA testing has been completed.
- Blocking defects have been resolved.
- Regression testing has been completed where applicable.
- Required business/product approvals have been obtained.
- Deployment prerequisites have been verified.
- Database and configuration changes have been reviewed.
- The release has been confirmed as production-ready.

5. Production Deployment Process

After successful QA/Staging validation and required approvals, the Release branch is merged into Master/Main.

The standard flow is:

Release → Master/Main → Production

The production deployment should be performed using the approved Master/Main version.

Before production deployment, the deployment owner should verify:

- Correct branch and commit are being deployed.
- Required approvals are completed.
- Production configuration is available and validated.
- Database scripts are reviewed and ready.
- Required environment variables/secrets are available.
- Deployment dependencies have been identified.
- Rollback or recovery procedures are available where required.
- Release version/build number is correct.
- Appropriate monitoring and post-deployment validation are planned.

After deployment, the application should be validated in the production environment to confirm that the release has been successfully deployed.

6. Hotfix Branch Strategy

A Hotfix branch is used to address a production defect that requires a fix before the normal development/release cycle can safely deliver the change.

Hotfixes are particularly important when the **Dev** and **Release** branches contain additional features or changes that are not yet approved for production.

In such cases, the production fix must not be based on Dev or Release because doing so could unintentionally introduce unreleased functionality into production.

6.1 Hotfix Branch Creation

A Hotfix branch must be created from the current **Master/Main** branch.

The flow is:

Master/Main → Hotfix

Example:

`hotfix/fix-production-notification`

The Hotfix branch should contain only the changes required to resolve the production issue, unless additional changes are explicitly approved as part of the emergency fix.

6.2 Hotfix Development Process

The standard Hotfix process is:

1. Identify and document the production issue.
2. Confirm that a Hotfix is required.
3. Create the Hotfix branch from the current production version of Master/Main.
4. Implement the minimum required fix.
5. Perform local/unit testing.
6. Raise a Pull Request for the Hotfix branch.
7. Deploy the Hotfix branch to QA/Staging where practical.
8. QA verifies the fix and performs appropriate regression testing.
9. Obtain the required approval.
10. Merge the approved Hotfix into Master/Main.
11. Deploy the approved Master/Main version to production.
12. Ensure the Hotfix change is also incorporated into Dev and any applicable Release branch to prevent the defect from being reintroduced in a future release.

6.3 Hotfix Synchronization

After a Hotfix is merged into Master/Main, the fix must be propagated back into the ongoing development line.

Depending on the repository state, this may be achieved by:

- Merging Master/Main into Dev.
- Cherry-picking the Hotfix commit into Dev.
- Applying the equivalent fix to Dev through a Pull Request.
- Updating an active Release branch where the fix is relevant.

The chosen approach should preserve the fix while minimizing the risk of unintentionally bringing unrelated production changes into development.

7. Branch Flow Summary

7.1 Normal Feature and Release Flow

The normal development lifecycle is:

Feature Branch

↓

Dev

↓

Release

↓

QA/Staging

↓

Master/Main

↓

Production

This ensures that development changes pass through integration and QA validation before reaching production.

7.2 Hotfix Flow

When a production issue must be fixed while Dev or Release contains unreleased changes:

Master/Main

↓

Hotfix Branch

↓

QA/Staging

↓

Master/Main

↓

Production

Afterward, the Hotfix must be propagated back into the development branch:

Hotfix → Dev

and, where applicable:

Hotfix → Active Release Branch

8. Pull Request Policy

Pull Requests (PRs) should be used to control the movement of code between branches.

8.1 General Requirements

- Developers should not directly push to protected branches such as Master/Main.
- Feature branches should be merged into Dev through Pull Requests.
- Release changes should be reviewed before being merged into Master/Main.
- Hotfixes should be reviewed before being merged into Master/Main.
- Required CI/CD checks must pass before a Pull Request is merged.
- Required reviewers should approve the Pull Request before merging.
- Review comments should be addressed before approval.
- Pull Requests should clearly describe the purpose and scope of the change.
- Related work items, tickets, or requirements should be referenced where applicable.

8.2 Pull Request Description

A Pull Request should provide sufficient information for reviewers and QA to understand the change.

Where applicable, include:

- Purpose of the change.
- Related ticket/work item.
- Features or defects addressed.
- Technical changes.
- Database changes.
- Configuration changes.
- API changes.
- Testing performed.
- Known limitations.
- Deployment considerations.
- Rollback considerations.

9. Commit Guidelines

Developers should create clear and meaningful commits.

Recommended practices

- Keep commits focused on a logical change.
- Use meaningful commit messages.
- Avoid committing unrelated changes together.
- Do not commit generated files unless required by the project.
- Do not commit secrets, passwords, API keys, certificates, tokens, or other sensitive information.
- Avoid committing temporary debugging code.
- Ensure code is formatted according to project standards before committing.
- Review the changes locally before pushing.

A good commit message should make it clear what was changed and, where appropriate, why it was changed.

10. Branch Naming Convention

Branches should follow a consistent naming convention so that their purpose can be identified easily.

Recommended format:

Feature:

`feature/<ticket-or-id>-<short-description>`

Example:

`feature/TAC-535-notification-update`

Bug Fix:

`bugfix/<ticket-or-id>-<short-description>`

Example:

`bugfix/ADF-536-pdf-generation`

Hotfix:

`hotfix/<ticket-or-id>-<short-description>`

Example:

`hotfix/ADF-542-production-notification`

Release:

`release/<version-or-release-name>`

Example:

`release/1.0.0`

The exact naming convention may be adjusted according to the repository's existing standards, but consistency should be maintained within each project.

11. Code Review Best Practices

Code review is a mandatory quality control step for changes entering shared or protected branches.

Reviewers should consider:

- Correctness of the implementation.
- Alignment with the requirement.
- Code readability and maintainability.
- Error handling.
- Security considerations.
- Performance implications.
- Database impact.
- API compatibility.
- Logging and monitoring requirements.
- Automated test coverage where applicable.
- Potential impact on existing functionality.
- Configuration and deployment requirements.

Reviewers should focus on the quality and risk of the change rather than only formatting or personal coding preferences.

12. Merge and Conflict Management

Developers are responsible for keeping their branches reasonably synchronized with the target branch.

Before merging a Feature branch into Dev:

- Ensure the Feature branch is based on an appropriate and reasonably current version of Dev.
- Resolve merge conflicts locally where possible.
- Run relevant tests after resolving conflicts.
- Verify that the conflict resolution has not changed the intended functionality.

Before promoting Release or Hotfix changes:

- Confirm that the correct commits are included.
- Verify that unrelated changes have not been introduced.
- Confirm that the resulting branch builds successfully.
- Perform the required validation after merging.

13. Environment and Branch Alignment

The following branch-to-environment relationship should normally be maintained:

Branch	Purpose	Environment
Feature	Individual development	Local/Developer
Dev	Integrated development	Development
Release	Release candidate and QA	QA/Staging
Master/Main	Production-ready code	Production
Hotfix	Production issue correction	QA/Staging → Production

The exact environment mapping may differ for individual projects, but the principle of keeping development, QA, and production code separated should be maintained.

14. Release Version and Traceability

Every production deployment should be traceable to a specific source code version.

Where applicable, the project should maintain:

- Release version.
- Build number.
- Git commit.
- Git tag.
- Deployment date/time.
- Related work items.
- Release notes.
- Database script/version.
- Configuration changes.

- Deployment approval.

This allows the team to identify exactly which source code was deployed to production and supports troubleshooting, auditing, and rollback activities.

15. Database Change Management

Database changes require additional care because they may not be automatically reversible.

For any release containing database changes:

- Identify all database changes before deployment.
- Maintain SQL scripts or migration files under source control.
- Ensure scripts are reviewed.
- Confirm the execution order of scripts, we are commiting the sql sripts into the project repo and while deployment we follow custom build approach for that have a discussion with your senior.
- Verify compatibility between the application version and database version.
- Identify whether a change is backward compatible.
- Identify any required data migration.
- Define rollback or recovery steps where applicable.
- Clearly communicate production database execution requirements.
- Ensure database changes are executed only by authorized personnel where required.

16. Configuration and Secret Management

Application configuration should be managed separately from source code where appropriate.

Developers must not commit:

- Passwords.
- API keys.
- Client secrets.
- Access tokens.

- Certificates/private keys.
- Connection strings containing credentials.
- Other sensitive production configuration.

Environment-specific configuration should be maintained through the approved configuration or secret-management mechanism.

Before deployment, the team should verify that all required configuration values exist for the target environment and have a discussion with your senior before committing such details.

17. Testing Requirements

Testing should be performed at multiple stages of the branching lifecycle.

Feature Development

Developers should perform appropriate:

- Unit testing.
- Local functional testing.
- Integration testing where applicable.

Dev

The integrated changes should undergo appropriate:

- Build validation.
- Automated tests.
- Integration testing.
- Basic regression validation.

Release

The Release branch should undergo formal QA/Staging testing, including appropriate:

- Functional testing.
- End-to-end testing.

- User acceptance testing where applicable.

Production

After deployment, appropriate smoke testing and production validation should be performed.

18. Developer Pre-Merge Checklist

Before raising a Pull Request, developers should verify the following:

- Correct source branch was used.
- Branch name follows the project convention.
- Changes are limited to the intended requirement.
- No unnecessary files or changes are included.
- Code has been formatted according to project standards.
- Local build succeeds.
- Relevant unit tests have been executed.
- Relevant functional tests have been completed.
- No debugging code remains.
- No secrets or sensitive information have been committed.
- Database changes are identified and documented.
- Configuration changes are identified and documented.
- Related ticket/work item is referenced.
- Pull Request description contains sufficient details.
- Merge conflicts have been resolved.
- CI/CD checks are passing.

19. Release Checklist

Before promoting a Release branch to Master/Main, verify:

- All intended features are included.
- Required bug fixes are included.
- QA testing is completed.
- No blocking defects remain.

- Required approvals are obtained.
- CI/CD checks are passing.
- Application build is successful.
- Database scripts have been reviewed.
- Configuration changes have been verified.
- Deployment dependencies are identified.
- Release version/build number is confirmed.
- Release notes are prepared where required.
- Rollback/recovery procedure is available where required.
- Production deployment plan is confirmed.

20. Hotfix Checklist

Before deploying a Hotfix to production, verify:

- Production issue has been identified and documented.
- Hotfix branch was created from the current Master/Main branch.
- The fix is limited to the required production issue.
- Local/unit testing has been completed.
- Pull Request review is completed.
- QA/Staging validation is completed where practical.
- Regression impact has been considered.
- Required approvals are obtained.
- Production deployment plan is confirmed.
- Rollback/recovery approach is understood.
- Hotfix is merged into Master/Main.
- The fix is propagated back to Dev.
- Relevant active Release branches are updated where required.
- Post-production validation is completed.

21. General Git Best Practices

The following practices should be followed across all repositories:

1. **Keep branches focused**

A branch should represent one feature, bug fix, hotfix, or clearly defined piece of work.

2. **Keep changes small and reviewable**

Smaller Pull Requests are easier to review, test, troubleshoot, and merge.

3. **Avoid long-lived Feature branches**

Long-lived branches increase the likelihood of merge conflicts and integration issues.

4. **Synchronize regularly**

Developers should regularly update Feature branches from Dev when appropriate.

5. **Protect important branches**

Dev, Release, and especially Master/Main should use branch protection policies where supported.

6. **Require Pull Request reviews**

Code should be reviewed before being merged into protected/shared branches.

7. **Use automated validation**

CI/CD pipelines should run builds, tests, code-quality checks, and other required validations before merging.

8. **Never commit secrets**

Sensitive credentials must always be stored using approved secret-management mechanisms.

9. **Keep production stable**

Master/Main should contain only code that has passed the required validation and approval process.

10. **Maintain traceability**

Every production change should be traceable to a Pull Request, commit, work item, release, or Hotfix.

11. **Do not use production as a development branch**

Development work should never be performed directly on Master/Main.

12. **Do not use Dev for emergency production fixes**

When Dev contains unreleased changes, production fixes should be based on Master/Main through the Hotfix process.

13. **Keep the branches synchronized after Hotfixes**

A production fix should be brought back into the development line to prevent the same defect from being reintroduced.

22. Branch Protection Recommendations

Where supported by the repository platform, the following branch protection policies are recommended:

Dev

- Require Pull Request for merge.
- Require successful CI/CD checks.
- Require code review.
- Prevent force pushes.
- Prevent branch deletion where appropriate.

Release

- Require Pull Request for changes.
- Require successful build and automated validation.
- Require appropriate QA/release approval.
- Prevent direct pushes.
- Prevent force pushes.

Master/Main

- Require Pull Request for all changes.

- Require multiple reviewers where appropriate.
- Require successful CI/CD checks.
- Require release/production approval where applicable.
- Prevent direct pushes.
- Prevent force pushes.
- Restrict branch deletion.
- Require all status checks to pass before merge.

23. Roles and Responsibilities

Developers

Developers are responsible for:

- Following the approved branching strategy.
- Creating the correct branch from the correct source branch.
- Writing and testing code before creating a Pull Request.
- Following commit and branch naming standards.
- Addressing code review comments.
- Resolving merge conflicts.
- Documenting deployment and configuration requirements.
- Ensuring Hotfix changes are propagated back to development branches.

Reviewers

Reviewers are responsible for:

- Reviewing the technical correctness of changes.
- Identifying potential defects and risks.
- Reviewing security and performance considerations.
- Confirming that the change is appropriate for the target branch.
- Ensuring required checks have passed before approval.

QA Team

QA is responsible for:

- Validating Release candidates in the QA/Staging environment.
- Performing functional and regression testing.
- Reporting defects clearly.
- Re-testing fixes.
- Providing release validation/approval according to the project process.

Release/Deployment Team

The Release/Deployment team is responsible for:

- Verifying deployment prerequisites.
- Deploying approved builds to the appropriate environments.
- Ensuring the correct branch/version is deployed.
- Executing or coordinating database and configuration changes.
- Performing post-deployment validation.
- Maintaining deployment traceability.

24. Exception Handling

Any deviation from this branching and release process should be explicitly discussed and approved by the relevant project or technical stakeholders.

Examples of potential exceptions include:

- Critical production incidents.
- Emergency security fixes.
- Infrastructure or platform limitations.
- Third-party deployment requirements.
- Temporary migration activities.

Even when an exception is required, the team should maintain traceability through Pull Requests, work items, approvals, and deployment records.

25. Branching Lifecycle Summary

The overall branching lifecycle can be summarized as follows:

Standard Development

Dev → Feature Branch → Dev

Multiple Feature branches may exist and be developed in parallel.

Release

Dev → Release → QA/Staging

The Release branch represents the release candidate being validated by QA.

Production

Release → Master/Main → Production

Only approved and validated releases should be promoted to production.

Production Hotfix

Master/Main → Hotfix → QA/Staging → Master/Main → Production

The Hotfix is created from the production code to avoid including unreleased Dev or Release changes.

Hotfix Synchronization

Master/Main/Hotfix → Dev

The production fix must subsequently be incorporated into the development line and applicable Release branches.

26. Guiding Principle

The primary principle of the Wiseliy branching strategy is:

Development should move forward without compromising production stability.

Dev is used for ongoing development, Feature branches provide isolated development, Release provides controlled QA/Staging validation, and Master/Main represents production-ready code.

When a production issue occurs, the Hotfix process allows the team to correct the production version independently of unreleased development work.

This separation provides a controlled path from development to production while maintaining traceability, code quality, collaboration, and production stability.